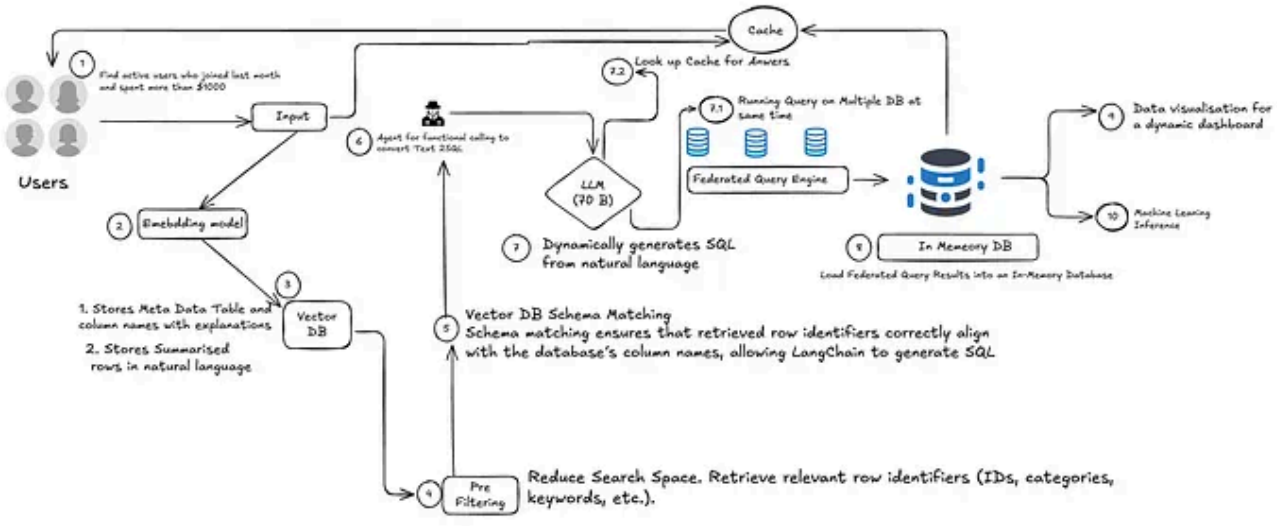
Part 2: Building a Scalable Text-to-SQL Agentic System with LangChain, Vector DB, and Multi DB Federated Queries

Indrajit Follow 6 min read · Mar 7, 2025



System Architecture

Introduction

Refer to previous blog to understand the problem Statement

In this blog, we will walk through a **complete implementation** of a **scalable Text-to-SQL system** using **LangChain for orchestration, a Vector DB (ChromaDB/FAISS) for schema matching & row summarization, and a Federated Query Engine (Trino/Presto) for querying multiple databases efficiently.**

This system allows users to ask **natural language queries**, which are then:

1. **Converted into embeddings** for schema & row retrieval.
2. **Pre-filtered using Vector DB** to reduce search space.
3. **Mapped to the correct schema using metadata embeddings.**
4. **Converted into an optimized SQL query** by LangChain.
5. **Executed across multiple databases** in parallel using a Federated Query Engine.
6. **Stored in an In-Memory Database** (DuckDB/Redis) for fast analytics & AI inference.

System Architecture

Key Components

User Query → Converts natural language input into embeddings.
Embedding Model → Embeds the query for semantic search **Vector DB** → Stores **table metadata** and **summarized row embeddings.**
Pre-Filtering → Reduces search space by retrieving relevant row identifiers.
Schema Matching → Ensures the correct column mappings.

LangChain (Orchestration) → Handles SQL generation, federated query execution, and caching.
LLM (GPT-4, 70B Model) → Generates optimized SQL queries.
Federated Query Execution → Runs SQL across multiple databases simultaneously.
In-Memory DB → Stores query results for quick analytics.
Data Visualization & Machine Learning → Uses query results for dashboards & AI models.

Step-by-Step Implementation

Install Dependencies

Before starting, install the necessary libraries:

```
pip install langchain openai asyncpg psycopg2 faiss-cpu chromadb redis trino duc
```

Convert User Query into Embeddings

We first **convert the user's natural language query into an embedding** using a **Sentence Transformer** model.

```
from sentence_transformers import SentenceTransformer
```

```
# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
# Function to convert user query into embedding
def embed_query(query):
    return model.encode(query)

# Example user query
query = "Find active users who joined last month and spent more than
$1000."
query_embedding = embed_query(query)
print(query_embedding)  # Output: Embedding vector
```

Pre-Filtering: Retrieve Relevant Rows from Vector DB

We **search the Vector DB** (ChromaDB/FAISS) for **relevant row embeddings** before running SQL queries.

```
import chromadb
```

```
# Initialize ChromaDB
client = chromadb.PersistentClient(path="./chroma_db")
row_collection = client.get_or_create_collection("row_data")

# Function to retrieve relevant rows based on query embedding
def retrieve_relevant_rows(query):
    query_embedding = embed_query(query)
    results = row_collection.query(
        query_embeddings=[query_embedding.tolist()],
        n_results=5
    )
    return [r["summary"] for r in results["metadatas"][0]]

# Example retrieval
filtered_rows = retrieve_relevant_rows(query)
print(filtered_rows)
```

This reduces the search space for SQL queries!

Schema Matching: Retrieve Correct Table & Column Mappings

We now **search the Vector DB** for **matching table/column names.**

```
# Initialize Vector DB for schema matching
schema_collection = client.get_or_create_collection("schema_metadata")
```

```
# Function to match schema
def match_schema(query):
    query_embedding = embed_query(query)
    results = schema_collection.query(
        query_embeddings=[query_embedding.tolist()],
        n_results=1
    )
    return results["metadatas"][0]["column_name"]

# Example Schema Matching
print(match_schema("active users count"))
```

Ensures SQL queries reference correct column names.

LangChain for SQL Generation & Federated Query Execution

We now **use LangChain to orchestrate SQL generation and execution.**

```
from langchain.chat_models import ChatOpenAI
from langchain.tools import Tool
from langchain.agents import initialize_agent, AgentType
```

```
# Initialize LLM
llm = ChatOpenAI(model="gpt-4-0613", temperature=0)

# SQL function generator
def generate_sql_query(intent: str, table: str, columns: list,
conditions: str = None):
    query = f"SELECT {', '.join(columns)} FROM {table}"
    if conditions:
```

```
        query += f" WHERE {conditions}"
    return query + ";"

# Register SQL tool in LangChain
sql_tool = Tool(
    name="SQL Generator",
    func=generate_sql_query,
    description="Generates SQL queries from user intent."
)

# Create LangChain Agent
agent = initialize_agent(
    tools=[sql_tool],
    llm=llm,
    agent=AgentType.OPENAI_FUNCTIONS,
    verbose=True
)

# Example Usage
generated_sql = agent.run(query)
print(generated_sql)  # Output: SQL Query
```

Now, LangChain dynamically generates SQL based on user input.

Execute SQL Federated Query Across Multiple Databases

Now, we **execute the generated SQL query across multiple databases using Trino/Presto.**

```
from trino.dbapi import connect
```

```
# Function to execute federated query
def execute_federated_query(sql_query):
    conn = connect(
        host="trino-coordinator",
        port=8080,
        user="admin",
        catalog="postgresql_db",
        schema="public"
    )
    cursor = conn.cursor()
```

```
    cursor.execute(sql_query)
    return cursor.fetchall()

# Run federated query
results = execute_federated_query(generated_sql)
print(results)
```

Executes queries across multiple databases simultaneously!

Store Results in In-Memory DB (DuckDB)

To avoid redundant queries, we **store the results in DuckDB.**

```
import duckdb
import pandas as pd
```

```
# Convert results to Pandas DataFrame
df = pd.DataFrame(results, columns=["user_id", "first_name",
"last_name", "total_spent"])

# Store in DuckDB for fast querying
conn = duckdb.connect(":memory:")
conn.register("federated_results", df)

# Run fast in-memory queries
query_result = conn.execute("SELECT * FROM federated_results WHERE
total_spent > 5000").fetchdf()
print(query_result)
```

DuckDB provides ultra-fast analytics & ML integration!

Benefits of the AI-Driven Text-to-SQL System

This **AI-powered Text-to-SQL system** brings together **LangChain, Vector Databases, Federated Query Engines, and In-Memory Processing**, offering numerous benefits in **speed, scalability, and accuracy.**

High-Performance Query Execution

Pre-Filtering with Vector DB → Reduces the search space before SQL execution, minimizing the need for full-table scans in PostgreSQL/MySQL.
Federated Query Execution → Runs SQL queries across multiple databases **simultaneously**, reducing query execution time.
In-Memory Processing (DuckDB/Redis) → Speeds up repeated queries by caching results.

Accurate Schema Matching & Query Generation

Vector DB Schema Matching → Ensures correct table/column selection, **preventing SQL errors.**
LangChain Function Calling → Dynamically **converts user input into optimized SQL queries.**

Example:

> **User Query:** "Find users who purchased red shoes in January."
> **Without Schema Matching:** The system may mistakenly reference `shoe_color` instead of `product_color`.

> **With Schema Matching:** Corrects the query to **match actual database column names.**

Eliminates incorrect queries and ensures accurate results!

Scalability & Multi-Database Support

Works with Multiple Databases → Supports **PostgreSQL, MySQL, Snowflake, BigQuery, and MongoDB.**
Handles Large-Scale Datasets → Federated queries ensure the system is **scalable for big data workloads.**

Example:
A company using **PostgreSQL for customers, MySQL for orders, and MongoDB for logs** can retrieve unified insights **without data migration.**

No need to centralize databases — query everything seamlessly!

Faster Query Execution with Caching & Precomputed Views

Redis for Caching → Stores frequently queried results, reducing redundant database hits.
Materialized Views in PostgreSQL → Precomputes expensive queries, making analytics faster.

Example:
If "**total active users per month**" is queried frequently, the result is stored in **Redis/DuckDB** for fast retrieval, instead of recalculating every time.

Reduces database load by up to 90%!

AI & Machine Learning Integration

Stored Query Results for AI Models → Data can be used for **predictive analytics, fraud detection, and recommendation systems.**
In-Memory Processing (DuckDB/Arrow) → Allows **fast training on precomputed query results.**

Example:

- A machine learning model predicting **customer churn** can use the retrieved SQL data directly.
- A **real-time fraud detection system** can analyze transactions from multiple databases **in seconds.**

This system supports AI-powered decision-making!

Real-Time Analytics & Dynamic Dashboards

Query results can be visualized in real-time dashboards (Streamlit/Power BI).
Supports complex aggregations & joins across multiple databases.

Example:
A financial firm tracking **"high-value transactions from new users"** can view **live reports in a dashboard** without waiting for batch processing.

Ideal for real-time business intelligence!

Fully Automated Workflow with LangChain

Handles complete orchestration (query generation, execution, and caching).
Integrates seamlessly with AI Agents for continuous improvement.

Example:

- Users **ask questions in natural language**, and the system **automatically retrieves answers.**
- AI agents continuously **optimize queries for better performance.**

No need for SQL expertise — automates the entire process!

Why Choose This System Over Traditional BI & SQL Approaches?

Feature	Traditional SQL Approach	Our AI-Driven System
Query Speed	Slow, full-table scans	✅ Pre-filtering & caching improve speed 10x
Database Support	Limited to one DB	✅ Federated queries across multiple DBs
Natural Language Processing	❌ Requires manual SQL writing	✅ Converts text to SQL automatically
AI & Machine Learning Integration	❌ Not optimized for ML workloads	✅ DuckDB & Arrow enable fast AI model training
Scalability	❌ Struggles with large datasets	✅ Distributed execution ensures scalability
Automation	❌ Requires human intervention	✅ Fully automated with LangChain

Final Thoughts

By integrating **LangChain, Vector DB, and Federated Queries**, this **AI-driven Text-to-SQL system** provides:

Understands natural language queries

10x faster query execution through pre-filtering & caching.
Accurate SQL generation with metadata schema matching.
Seamless multi-database querying using a federated engine.
Support for AI & real-time analytics with in-memory processing.

Uses vector search for row pre-filtering
Performs schema matching for correct SQL generation
Executes federated queries across multiple databases
Stores results in an in-memory DB for fast analytics

This architecture is ideal for large-scale AI-driven analytics!

Text2sql | LIm Applications | Generative Ai Solution | Generative Ai Use Cases | Agentic Ai

Written by Indrajit

66 followers · 1 following

Follow